

FEBAUTO Racing: Competitor Setup (rough)

macOS, from nothing to a driving car

Formula Electric at Berkeley, Autonomous Team

September 2026 · draft, not for distribution

What you are setting up

A simulator of the RoboRacer 1:10 car runs natively on your Mac. Your code runs in a Docker container that carries ROS 2 and the bridge to the simulator. You never install ROS on the Mac. Foxglove shows you the car's sensors live.

feb-racing/	the repo, your clone
feb-sim	the launcher
stack/	YOUR code (gitignored)
starter_kit/feb_driver/	the template you copy into stack/
tracks/	track folders
~/feb-sim/app/mac/	the simulator app, downloaded by setup
Docker Desktop	runs the devkit container

The loop: the container mounts `stack/` and builds it; the bridge turns the simulator's stream into ROS 2 topics under `/autodrive/roboracer_1/`; your node reads the sensors and publishes two floats, `throttle_command` and `steering_command`, both in `[-1,1]`; the bridge sends them to the simulator; the car moves.

1 Install, once

1. **Docker Desktop.** Install with `brew install --cask docker-desktop` (or download from <https://docker.com>), then `open -a Docker`. Do not install `docker/tap/sbx`; that is Docker's agent sandbox tool, not Docker. On first launch it may report "Failed to install Rosetta": install Rosetta on the Mac first, `softwareupdate --install-rosetta --agree-to-license`, then relaunch Docker Desktop and let it retry. Our images run natively on Apple Silicon, so Rosetta is only for other x86 images later. Check with `docker run --rm hello-world`. Leave the app running. It is the whale icon in the menu bar. On a Mac, containers live in a small Linux virtual machine that exists only while the app is open, so `docker` commands fail with "cannot connect to the Docker daemon" if you quit it. Idle it costs almost nothing; tick *Start Docker Desktop when you sign in* in its settings. (On Linux, Docker is a background service that starts at boot, which is why nothing has to be opened there.)
2. **Foxglove.** Download from <https://foxglove.dev/download>, open once, sign in with a free account.
3. **GitHub CLI.** In Terminal: `brew install gh`, then `gh auth login` and follow the prompts.
4. **Python 3.** Already on the Mac. If Terminal says otherwise: `xcode-select --install`.

2 Clone and set up

```
git clone https://github.com/Pranman1/feb-racing
cd feb-racing
./feb-sim setup --team "Your Team"
```

Pulls the devkit image and downloads the Mac app into `~/.feb-sim/app/`. A few minutes. If the app later refuses to open (“damaged” or “cannot be opened”), it is unsigned; run once:

```
xattr -dr com.apple.quarantine ~/.feb-sim/app/mac/"FEB Simulator.app"
codesign --force --deep -s - ~/.feb-sim/app/mac/"FEB Simulator.app"
```

3 Drive manually

```
./feb-sim run
```

Arrow keys drive. Menu top-left: Manual/Autonomous, Track, Cars, Look, camera. Close the window to stop. Nothing drives by itself without a stack.

4 Drive autonomously with the starter kit

```
cp -r starter_kit/feb_driver stack/
./feb-sim run --stack "ros2 launch feb_driver drive.launch.py"
```

The container builds the package, the simulator opens, the car drives, Foxglove opens on its topics. The driver is `stack/feb_driver/feb_driver/reactive_driver.py`; its numbers are in `stack/feb_driver/config/driver.yaml`. Edit, save, run the same command again.

5 Score and post

```
./feb-sim practice --stack "ros2 launch feb_driver drive.launch.py"
./feb-sim submit
```

6 Looking inside: extra terminals and Foxglove

More terminals into the devkit

The container keeps running while the simulator is open. Open as many Terminal tabs as you like and, in each, from the repo folder:

```
./feb-sim shell
```

That drops you into a ROS 2 shell inside the container, with the environment already sourced. Useful commands there:

```
ros2 topic list                                # everything on the graph
ros2 topic hz /autodrive/roboracer_1/lidar      # is the sensor stream alive, at
  what rate
ros2 topic echo /autodrive/roboracer_1/throttle_command # what your node is sending
ros2 topic echo --once /autodrive/roboracer_1/imu
ros2 node list                                  # your node should be here
ros2 node info /driver
```

```
ros2 param list /driver          # live parameters
ros2 bag record -a -o /home/autodrive_devkit/src/stack/mybag # lands in your stack/
                                folder on the Mac
```

`exit` leaves the shell; the container keeps running. `docker exec -it feb-devkit bash` is the same thing without the launcher. Files you write under `/home/autodrive_devkit/src/stack/` inside the container appear in `stack/` on the Mac, and the other way round.

Foxglove

Installed in step 2 of the install list; sign in once. After that, `./feb-sim run` opens Foxglove connected to your car a few seconds after the simulator window. If it does not, or you want a second window: in Foxglove choose *Open connection, Foxglove WebSocket*, address `ws://localhost:8765`.

Panels worth adding (the `+` at the top right of a panel):

- **3D**: add the topic `/autodrive/roboracer_1/lidar`; set the display frame to the lidar's frame. The scan appears as points around the car.
- **Image**: `/autodrive/roboracer_1/front_camera`.
- **Plot**: `throttle_command.data` and `steering_command.data` against time; add `left_encoder.position` to watch the wheel.
- **Raw Messages**: any topic, to read exact values.
- **Topics** (left sidebar): every topic with its rate; a dash means nothing is publishing.

Save the layout (*Layout, Save*) and it comes back next time. Foxglove also opens the bags that practice records under `runs/<id>/`, and any bag you record yourself: *Open local file*.

7 When something goes wrong

```
docker logs feb-devkit          # build errors, your node's output
./feb-sim shell                 # a ROS 2 terminal inside the container
./feb-sim stop                  # kill everything and start over
```

Nothing drives: correct without `-stack`; with one, read `docker logs`. Port 4567 in use: `./feb-sim stop`. Docker pull denied: Docker Desktop is not running.

8 A bare-bones driver from scratch

The smallest ROS 2 package that drives the car: it steers toward the deepest lidar beam. It has no safety bubble and will hit walls in corners; the starter kit is the real thing. Create `stack/my_driver/` with these files.

`stack/my_driver/package.xml`

```
<?xml version="1.0"?>
<package format="3">
  <name>my_driver</name>
  <version>0.0.1</version>
  <description>Bare-bones driver</description>
  <maintainer email="you@berkeley.edu">you</maintainer>
  <license>MIT</license>
  <exec_depend>rclpy</exec_depend>
  <exec_depend>sensor_msgs</exec_depend>
  <exec_depend>std_msgs</exec_depend>
```

```
<export><build_type>ament_python</build_type></export>
</package>
```

stack/my_driver/setup.py

```
from setuptools import setup
package_name = "my_driver"
setup(
    name=package_name, version="0.0.1", packages=[package_name],
    data_files=[
        ("share/ament_index/resource_index/packages", ["resource/" + package_name]),
        ("share/" + package_name, ["package.xml"]),
        ("share/" + package_name + "/launch", ["launch/drive.launch.py"]),
    ],
    install_requires=["setuptools"], zip_safe=True,
    entry_points={"console_scripts": ["driver = my_driver.driver:main"]},
)
```

stack/my_driver/setup.cfg

```
[develop]
script_dir=$base/lib/my_driver
[install]
install_scripts=$base/lib/my_driver
```

Two empty files

stack/my_driver/resource/my_driver and stack/my_driver/my_driver/__init__.py.

stack/my_driver/my_driver/driver.py

```
import math
import numpy as np
import rclpy
from rclpy.node import Node
from rclpy.qos import QoSProfile, QoSReliabilityPolicy, QoSHistoryPolicy
from sensor_msgs.msg import LaserScan
from std_msgs.msg import Float32

NS = "/autodrive/roboracer_1/"
QOS = QoSProfile(reliability=QoSReliabilityPolicy.RELIABLE,
                 history=QoSHistoryPolicy.KEEP_LAST, depth=10)
MAX_STEER = 0.5236 # rad; the command is normalised to [-1, 1]

class Driver(Node):
    def __init__(self):
        super().__init__("driver")
        self.throttle = self.create_publisher(Float32, NS + "throttle_command", QOS)
        self.steering = self.create_publisher(Float32, NS + "steering_command", QOS)
        self.create_subscription(LaserScan, NS + "lidar", self.on_scan, QOS)

    def on_scan(self, scan):
        ranges = np.asarray(scan.ranges, dtype=float)
```

```

    ranges[~np.isfinite(ranges)] = scan.range_max
    angles = scan.angle_min + np.arange(len(ranges)) * scan.angle_increment
    ahead = np.abs(angles) < math.radians(90)          # front half only
    target = angles[ahead][np.argmax(ranges[ahead])]    # bearing of the deepest beam
    steer = float(np.clip(target / MAX_STEER, -1.0, 1.0))
    speed = 0.08 if abs(steer) < 0.3 else 0.05          # 0 is a hard brake: never
    send it while moving
    self.steering.publish(Float32(data=steer))
    self.throttle.publish(Float32(data=speed))

def main():
    rclpy.init()
    node = Driver()
    try:
        rclpy.spin(node)
    except KeyboardInterrupt:
        pass
    node.destroy_node()
    rclpy.shutdown()

```

`stack/my_driver/launch/drive.launch.py`

```

from launch import LaunchDescription
from launch_ros.actions import Node

def generate_launch_description():
    return LaunchDescription([
        Node(package="my_driver", executable="driver", name="driver",
            parameters=[{"use_sim_time": True}], output="screen", emulate_tty=True),
    ])

```

Run it

```
./feb-sim run --stack "ros2 launch my_driver drive.launch.py"
```

9 What the car gives and takes

Topic (under <code>/autodrive/roboracer_1/</code>)	Meaning
<code>lidar</code>	LaserScan, 1080 beams over 270°, 0.06 to 10 m
<code>imu</code>	orientation, angular velocity, linear acceleration
<code>left_encoder, right_encoder</code>	wheel angle in rad; radius 0.059 m
<code>front_camera</code>	image
<code>throttle_command, steering_command</code>	you publish these: Float32 in $[-1, 1]$; throttle 0 is a hard brake; steering $\pm 1 = \pm 0.5236$ rad
<code>ips, odom, lap and collision counters</code>	ground truth: training only, barred at race time

Publish as many topics of your own as you like; only the two commands reach the car. Use message stamps or `use_sim_time`, never the wall clock, for speeds and integrals. Throttle about 0.1 is roughly 2 m/s.